



SOA & Microservices

Filière : Génie Informatique

TP – Microservice de gestion de produits avec Eclipse MicroProfile sur Quarkus

Présenté par :

IBOURK Khadija
ABOUTOFIL Salma
Mohammed Reda Kadiri
Nabil Abdelwahed

DELLAHI Hiba
Fikhane Nizar
Souhail Mossalim

Encadré par :

Prof. Bouchra KAROUM

Année Universitaire 2025 – 2026

Objectifs pédagogiques

À l'issue de ce TP , vous saurez :

- Créer un microservice REST ultra-performant avec Quarkus
- Externaliser entièrement la configuration avec MicroProfile Config
- Mettre en œuvre la résilience avec MicroProfile Fault Tolerance (Circuit Breaker, Retry, Timeout, Fallback)
- Exposer et tester des Health Checks (Liveness & Readiness)
- Exposer des métriques Prometheus et des traces OpenTelemetry
- Sécuriser des endpoints avec JWT et RBAC (rôles)

Contexte métier

Vous devez réaliser le microservice **product-service** qui gère le catalogue produits d'une boutique en ligne. Les données seront stockées en mémoire (pas de base de données).

1 Partie 1 – Microservice de base et configuration externalisée

1. Générer le projet sur <https://code.quarkus.io> avec :

- Group : `com.microprofile.tp`
- Artifact : `product-service`
- Build tool : Maven

Ajouter toutes les extensions nécessaires au fur et à mesure du TP.

2. Créer la classe `Product` avec les attributs suivants :

- `id` : `Long`
- `name` : `String` (obligatoire)
- `description` : `String`
- `price` : `double ( 0)`

Ajouter les annotations de validation Jakarta Bean Validation appropriées.

3. Implémenter une API REST complète sous le chemin de base `/products` avec les opérations CRUD suivantes :

- `GET /products` → liste tous les produits
- `GET /products/{id}` → détail d'un produit
- `POST /products` → création (retour 201 + produit avec id)
- `PUT /products/{id}` → mise à jour complète
- `DELETE /products/{id}` → suppression

4. Externaliser la configuration dans `application.properties` :

- `product.management.vat-rate` → taux de TVA en %
- `product.promotion.prefix` → préfixe promo (ex : "[PROMO] ")
- `app.environment` → environnement courant (dev/test/prod)

Injecter ces valeurs avec `@ConfigProperty` dans la ressource.

5. Créer un endpoint `GET /products/config-check` qui retourne en JSON les valeurs injectées ainsi que le prix TTC d'un produit à 100 € HT.

2 Partie 2 – Résilience, Observabilité et Sécurité

6. **Tolérance aux pannes (Fault Tolerance)** Ajouter un endpoint `GET /products/simulate-external-call` qui simule un service tiers instable (échoue 70 % du temps). Appliquer les 4 mécanismes suivants :

- `@Retry` (4 tentatives max)
- `@Timeout` (1 seconde)
- `@CircuitBreaker` (ouvre après 4 requêtes dont 50 % d'échecs)
- `@Fallback` retournant une réponse par défaut

7. **Health Checks** Implémenter :

- Un check `Liveness`
- Un check `Readiness` qui vérifie qu'il y a au moins un produit dans le catalogue

Les endpoints `/q/health/live`, `/q/health/ready` et `/q/health` doivent retourner UP.

8. **Métriques et Observabilité**

- Ajouter au moins une métrique personnalisée (counter et timer) sur une méthode CRUD
- Vérifier l'exposition sur `/q/metrics`

9. **Sécurité avec JWT**

- Générer une paire de clés RSA (`private.pem` et `public.pem`)
- Configurer Quarkus pour valider les JWT avec la clé publique
- Créer un endpoint `GET /products/admin/stats` accessible **uniquement** aux utilisateurs ayant le rôle `admin`